Rapport de Révision de la Modélisation UML

Projet Java : ZooKeeper

Équipe de Développement

23 octobre 2026

1 Introduction et Contexte

Dans le cadre du projet **ZooKeeper**, une révision critique du diagramme de classes initial a été effectuée. L'objectif est d'aligner le modèle conceptuel avec les exigences du cahier des charges tout en appliquant les principes fondamentaux de la programmation orientée objet (POO), notamment le principe de responsabilité unique (*Single Responsibility Principle - SRP*) et la robustesse du typage.

2 Modifications Apportées et Justifications

2.1 1. Introduction des Énumérations (`enum`)

Dans la première version, les attributs de régime alimentaire et d'état de santé étaient modélisés par des chaînes de caractères (`String`).

— `Regime` : `CARNIVORE`, `HERBIVORE`, `OMNIVORE`.

— `StatutSante` : `BONNE_SANTE`, `MALADE`, `EN_SOIN`, `VACCIN_A_RENOUVELER`.

Justification : Cette typification forte élimine tout risque d'erreur de frappe (ex : « carnivor »), garantit l'intégrité des données lors du contrôle de compatibilité dans les enclos et simplifie la génération des alertes sanitaires.

2.2 2. Refonte de la Hiérarchie du Personnel (`Employe`)

Le modèle initial attribuait indistinctement l'entretien des enclos et les soins médicaux à la seule classe `Soigneur`. Une refonte a été opérée :

— **Classe Abstraite** `Employe` : Factorise les attributs communs (`nom`, `matricule`).

— **Classe** `Soigneur` : Spécialisée dans la gestion quotidienne des enclos (`assignerEnclos`) et l'alimentation (`planifierRepas`). Elle conserve la relation d'association avec la classe `Enclos`.

— **Classe** `Veterinaire` : Spécialisée dans le suivi médical (`planifierSoin`, `examiner`). Elle interagit directement avec les classes `Animal` et `Soin`.

Justification : Respect strict de la séparation des responsabilités. Le vétérinaire n'a pas à intervenir sur la logistique des enclos, et le soigneur n'effectue pas d'actes vétérinaires.

2.3 3. Réajustement des Interfaces (`Volant`, `Nageur`, `Soignable`)

L'implémentation aveugle de toutes les interfaces par toutes les familles d'animaux a été corrigée.

— L'interface `Soignable` reste portée par la classe abstraite `Animal`.
— Les interfaces de capacité motrice (`Volant`, `Nageur`) sont affectées avec discernement pour éviter qu'un mammifère terrestre n'hérite d'une méthode `voler()`.

3 Perspective : Intégration JavaFX et Scene Builder

L'ensemble de cette couche objet constitue le **Modèle (Model)** au sein de l'architecture MVC. L'interface graphique conçue sous *Scene Builder* (fichiers FXML) et gérée sous *Eclipse* viendra consommer ces classes métier sans nécessiter de refonte structurelle.